



Universidade Federal do Ceará - Campus de Quixadá

TRABALHO I
Compiladores
Prof. Lucas Ismaily

INFORMAÇÕES IMPORTANTES

O Trabalho I contém três opções para escolher. As opções são mutuamente exclusivas, isto é, cada equipe pode escolher **exatamente uma opção**. A data máxima de entrega do trabalho é **01/06/2026**. Porém, recomendo fortemente que entreguem antes, para evitar imprevistos. **Atenção:** findado o prazo de envio, todos os grupos que não enviaram receberão automaticamente nota **zero**. A entrega será **somente** via e-mail (ismailybf@ufc.br, assunto: Trabalho I - Compiladores), numa pasta zipada contendo todos os arquivos, e se preciso, instrução para execução. Também deve conter um arquivo contendo todos os nomes do membros da equipe.

Cada equipe **deve conter no mínimo 3 e máximo 5 alunos**. Sejam honestos com vocês e comigo. Qualquer fraude será punida com nota zero para todos os envolvidos.



PARA AS OPÇÕES 1 E 3 DO TRABALHO CONSIDERE A LINGUAGEM LangC

A **LangC** é uma linguagem minimalista, declarativa e estruturada, inspirada em C. Ela possui sintaxe simples, poucas palavras reservadas e regras rígidas.

1. Alfabeto da linguagem

A linguagem aceita:

- letras do alfabeto português sem acentuação: a-z, A-Z
- dígitos: 0-9
- espaço, tabulação e quebra de linha
- símbolos reservados da linguagem: + - * / = > < () ; " _ ! @ # \$ % & ? |

2. Tipos de dados suportados

A LangC possui apenas **três categorias de valores**:

2.1. Tipo num

Representa números inteiros. Exemplo:

```
num idade = 25;
```

2.2. Tipo text

Representa cadeias de caracteres entre aspas duplas. Exemplo:

```
text nome = "Lucas";
```

2.3. tipo bool

A linguagem aceita os literais:

- true
- false

Esses valores podem ser usados em expressões relacionais e exibidos com show. Exemplo:

```
show 5 > 2;
```

3. Palavras reservadas

As palavras reservadas da LangC são:

- num
- text
- show
- true
- false



- bool

4. Identificadores

Nomes de variáveis devem seguir estas regras:

- podem conter letras, números e _
- devem começar com letra ou _
- não podem começar com: ! @ # \$ % & ? / |
- não podem ter mais de **30 caracteres**
- não podem ser iguais a palavras reservadas

Exemplos válidos:

x
idade
_nome
valor1
mensagem_final

Exemplos inválidos:

1valor
@teste
show
num
nome-completo

5. Operadores suportados

5.1 Operadores aritméticos

- +
- -
- *
- /

5.2 Operadores relacionais

- >
- <
- =
- ==

Na LangC, o símbolo = tem dois usos, conforme o contexto:



- **atribuição**, quando aparece sozinho em declaração:

```
num a = 10;
```

- **comparação**, quando aparece duplicado em uma expressão:

```
show a == 10;
```

6. Estrutura do programa

Um programa é composto por uma sequência de instruções.

Regras:

- toda instrução deve terminar com ;
- não existem blocos { }
- não existem funções
- não existem laços
- não existem condicionais
- não existe entrada de dados

7. Instruções da linguagem

A LangC possui apenas dois tipos de instrução:

7.1 Declaração de variável

Formato:

```
tipo identificador = valor_ou_expressao;
```

Exemplos:

```
num x = 10;  
num y = 5 + 2;  
text msg = "Ola";  
bool eq = 8 == 3;
```

7.2 Saída com show

Formato:

```
show expressao;
```

Exemplos:

```
show x;  
show "Oi";
```



```
show 2 > 1;  
show 5 + 3;
```

8. Regras semânticas

8.1 Tipo num

Aceita:

- números inteiros
- operações entre números inteiros

Exemplos válidos:

```
num a = 5;  
num b = a + 3;  
num c = 10 / 2;
```

Exemplos inválidos:

```
num a = "texto";  
num b = true;
```

8.2 Tipo text

Aceita apenas cadeias entre aspas duplas.

Exemplo:

```
text nome = "Maria";
```

Exemplo inválido:

```
text nome = 10;
```

8.3 Valores booleanos

São resultados de comparações ou literais true e false.

Exemplos:

```
show 2 > 1;  
show true;  
show false;
```

9. Precedência de operadores

Para simplificar a interpretação:

1. parênteses ()



2. multiplicação e divisão * /
3. soma e subtração + -
4. comparação > < =

Exemplo:

```
show 2 + 3 * 4;
```

Resultado:

14

Exemplo:

```
show (2 + 3) * 4;
```

Resultado:

20

10. Exemplo de programa em LangC

```
show 2 > 2;  
num a = 5;  
num b = 10;  
num soma = a + b;  
text mensagem = "Oi!";  
show mensagem;  
show a;  
show soma;  
show a < b;  
show a = 5;
```

Saída esperada

```
false  
Oi!  
5  
15  
true  
true
```



Opção 1 – Análise Léxica

1. (2,0 pontos) Crie *Tokens* apropriados e para cada *Token* faça uma Expressão Regular para a Linguagem **LangC**.
2. (2,0 pontos) Implemente um algoritmo que recebe como entrada todas as Expressões Regulares da Questão anterior e retorna um único Autômato Finito Não-Determinístico (NFA).
3. (3,0 pontos) Implemente um algoritmo que recebe como entrada um Autômato Finito Não-Determinístico (NFA) e retorna um Autômato Finito Determinístico (DFA). A forma de representação dos Autômatos é livre, ou seja, você pode representá-los como matriz, lista, dicionário etc.
4. (3,0 pontos) Utilizando o DFA da Questão 3, implemente um analisador léxico para a Linguagem **LangC**. Além do código, é preciso entregar um arquivo .txt contendo a lista de *tokens* utilizados e o que eles representam. O arquivo tem o seguinte formato: cada linha contém duas informações separadas por espaço, sendo a primeira posição o *token* e a segunda o que ele representa. Se o *token* representa mais de uma entidade, separe-os por vírgula.

Entrada

A entrada é composta por um código fonte de um programa qualquer escrito em **LangC**.

Saída

Para cada entrada, seu programa deve produzir uma sequência de *Tokens* ou a palavra ERRO, caso a entrada tenha erro léxico.

Exemplo

Entrada

```
num a = 0 ;  
num b = 5 + a ;  
text c = "teSte" ;
```

Saída

```
NUM VAR EQ NUM SEMICOLON  
NUM VAR EQ NUM ADD VAR SEMICOLON  
TEXT VAR EQ CONST SEMICOLON
```



Opção 2 – Análise Sintática

1. Dada a gramática LR(0) da Figura 1. Você deve Implementar:
 - I. (3,5 pontos) um algoritmo que calcula os conjuntos FISRT e FOLLOW.
 - II. (3,5 pontos) um algoritmo que constrói o Autômato LR(0).
 - III. (3,0 pontos) um algoritmo para o reconhecimento sintático. Isto é, dada uma palavra w , o seu analisador deve ser capaz de dizer se w obedece ou não as regras da gramática.

$$\begin{array}{ll} 0 & S' \rightarrow S\$ \\ 1 & S \rightarrow (L) \\ 2 & S \rightarrow x \\ 3 & L \rightarrow S \\ 4 & L \rightarrow L , S \end{array}$$

Figura 1: Gramática LR(0) para ser utilizada.

Entrada

A entrada é composta por um código fonte de um programa qualquer escrito na gramática escolhida.

Saída

Para cada entrada, seu programa deve produzir uma mensagem de “Sucesso” ou exibir um erro sintático.

Exemplo.

Entrada

Teste 1. id + id

Teste 2. id ** id

Saída

Teste 1. Sucesso

Teste 2. Erro sintático



Opção 3 – Análise Semântica

1. Implemente um analisador semântico para a Linguagem **LangC**. O seu analisador semântico deve verificar:
 - I. (3,5 pontos) se as operações usam tipos compatíveis;
 - II. (3,0 pontos) se as variáveis estão sendo usadas na ordem correta e verificar escopo;
 - III. (3,5 pontos) se as variáveis estão sendo usadas sem serem declaradas.

Entrada

A entrada é composta por um código fonte de um programa qualquer escrito em **LangC**

Saída

Para cada entrada, seu programa deve produzir uma mensagem de “Sucesso” ou um erro semântico.

Exemplo

Entrada

```
int a = 3 ;  
int b = 5 ;  
int c = a + b ;
```

Saída

Sucesso